

Spaces, Duals, Morphisms, and BlockTensor

August 22, 2026

Abstract

This note refines the local tensor model in *Planar Tensor Networks, Braiding, and Interface Invariants*. Concrete Uni20 values now model the `Space` concept directly: `BlockSpace`, `IrregularSpace`, `LocalSpace`, `QNumSpace`, and `DenseSpace`. Ordered `Domain<...>` and `Codomain<...>` values are also implemented. Space structural data is immutable, while each leg space carries a mutable string label used by exact compatibility checks. There is no wrapper such as `Space<LocalSpace>` and no globally generated space identifier. The categorical distinction between a space and its dual remains independent of the distinction between a morphism's domain and codomain. This model reduces to signed charge book-keeping for bosonic $U(1)$, while retaining the boundary data needed by later non-Abelian and braided implementations.

Contents

1	Relationship to the Planar-Network Design	1
2	Concrete Space Values	1
2.1	Structural immutability	2
2.2	Labels and exact equality	2
2.3	Why there is no global identifier	2
3	Object Duality and Morphism Side	3
4	All-Out Boundary and Block Selection	3
5	Repartition Is Wire Bending	4
6	Composition and Partial Contraction	4
7	BlockTensor Consequences	4
8	Storage Orthogonality	5
9	Decompositions Create Bond Values	5
10	Initial $U(1)$ Implementation Sequence	5
11	Interface Invariants	6
11.1	Required tests	6
12	Conclusion	7

1 Relationship to the Planar-Network Design

The earlier planar-network note established that:

1. a tensor stores a local ordered domain and codomain;
2. a network or contraction plan stores global embedding and crossing information;
3. contraction lowers checked compositions, recouplings, braids, and traces to block worklists and numerical kernels.

Those conclusions remain. This note refines the local boundary vocabulary and removes two unnecessary ideas: a templated `Space<...>` ownership wrapper and a globally coordinated `SpaceId`.

Two categorical questions remain independent:

1. Is the object X or its dual $X^\vee$?
 2. Does this occurrence appear in the domain or codomain?
- A domain and a codomain may each contain primal or dual objects.

2 Concrete Space Values

`Space` is a C++ concept. A concrete model has copyable value semantics, exact equality, a string label, and an operation for replacing that label. Five models establish the initial vocabulary:

Type	Structural data	Typical role
<code>BlockSpace</code>	canonical unique (q, d_q) sectors	MPS virtual bond
<code>IrregularSpace</code>	ordered (q, d_q) blocks, including repeats	projected bond before repacking
<code>LocalSpace</code>	ordered local-state charges	physical site
<code>QNumSpace</code>	one irrep label	irreducible operator leg
<code>DenseSpace</code>	symmetry-neutral extent	batch or bundled tensor axis

`BlockSpace`, `IrregularSpace`, `LocalSpace`, and `QNumSpace` also model `SymmetrySpace`. `DenseSpace` deliberately does not: it contributes no quantum number to the selection rule.

2.1 Structural immutability

Sector content, local-state order, irrep, and dense extent are fixed at construction. A tensor can therefore retain a space value without its shape or block interpretation changing behind the storage map.

`BlockSpace` requires an explicit valid symmetry even when empty. It accepts a range of (q, d_q) sectors, requires positive dimensions and matching symmetries, sorts sectors canonically, and rejects duplicate quantum numbers.

`IrregularSpace` also requires an explicit valid symmetry, but preserves the supplied block order and permits repeated quantum numbers. It is useful when projecting out part of a symmetry causes previously distinct sectors to acquire the same remaining charge. Regularization coalesces those blocks into a canonical `BlockSpace` and records the destination block and degeneracy range of every original segment.

`LocalSpace` also requires an explicit valid symmetry. It preserves input order and repeated charges because different basis states may transform in the same sector. Explicit regularization coalesces those states into a `BlockSpace` and records each state's block index and within-block offset.

`QNumSpace` carries one valid `QNum`. Its symmetry follows from that value. `DenseSpace` carries only a nonnegative extent.

2.2 Labels and exact equality

Every concrete space carries a `std::string` label. The empty string is a valid unlabelled value. Labels need not be unique, interned, generated, or coordinated between MPI ranks.

Exact equality compares both structure and label:

$$X = Y \iff \text{structure}(X) = \text{structure}(Y) \wedge \text{label}(X) = \text{label}(Y).$$

Changing a label does not alter extent, sectors, local-state order, or stored tensor data. It does change exact space compatibility. A network operation which labels a bond must update both endpoint leg occurrences. For example, an MPS may initialize all spaces and then assign labels "0", "1", and so on to the corresponding bond endpoints.

A future structural relation such as `isomorphic_space(X,Y)` may deliberately ignore labels. Algorithms must choose exact equality or structural compatibility explicitly.

2.3 Why there is no global identifier

A global identifier would require generation, serialization, and distributed agreement. It would also make independently reconstructed but intentionally compatible spaces unequal unless the identifier were threaded through every construction path.

The required semantics are already represented by immutable structure and a user-controlled label. Values are naturally copyable and serializable, and an operation may request either exact equality or an explicit structural conversion.

3 Object Duality and Morphism Side

A symmetry-aware tensor is a morphism

$$A : X_0 \otimes \cdots \otimes X_{m-1} \longrightarrow Y_0 \otimes \cdots \otimes Y_{n-1}.$$

Its local boundary is the ordered pair

$$\text{Domain}[X_0, \dots, X_{m-1}], \quad \text{Codomain}[Y_0, \dots, Y_{n-1}].$$

The implemented `Domain<Spaces...>` and `Codomain<Spaces...>` templates are distinct heterogeneous value types. They retain their concrete space types and ordered values, expose space structure read-only, and permit an individual label to be changed with `set_label<I>()`. Exact equality includes those labels. Empty boundaries represent the tensor unit.

The current factor vocabulary contains concrete `Space` values. A future generic `Dual<S>` adaptor may extend it once the required categorical operations are established. Parallel concrete families such as `DualBlockSpace` and `DualLocalSpace` are unnecessary unless a future category demonstrates that a generic adaptor cannot express its semantics.

Duality is involutive:

$$(X^\vee)^\vee \cong X.$$

For $U(1)$, duality negates charge. A richer category also owns the evaluation, coevaluation, pivotal, normalization, and braid data required to use dual objects.

Every boundary entry is one ordered leg occurrence. Several occurrences may carry equal copies of the same space value. Position, morphism side, object duality, and label remain distinct properties.

4 All-Out Boundary and Block Selection

Selection rules are easiest to state after orienting every boundary leg outward:

- a codomain factor X contributes X ;
- a domain factor X contributes $X^\vee$.

Dualizing a tensor product reverses order. One canonical all-out ordering is

$$X_{m-1}^\vee \otimes \cdots \otimes X_0^\vee \otimes Y_0 \otimes \cdots \otimes Y_{n-1}.$$

For bosonic $U(1)$, a block is legal when

$$\sum_j q(Y_j) - \sum_i q(X_i) = 0.$$

For an MPS site

$$A : L \otimes P \longrightarrow R,$$

this is the familiar

$$q_R = q_L + q_P.$$

The generic implementation asks the category for invariant channels; it does not hardcode charge addition. Multiplicity-free $U(1)$ returns zero or one channel, while non-Abelian categories may return several channels and a non-empty coupling descriptor.

5 Repartition Is Wire Bending

Moving a factor between domain and codomain is not a raw metadata edit. It is an explicit wire-bending operation, provisionally `repartition`, based on rigid-category isomorphisms such as

$$\text{Hom}(A \otimes X, B) \cong \text{Hom}(A, B \otimes X^\vee).$$

The move toggles object duality and may reverse order when several factors move:

$$(X \otimes Y)^\vee \cong Y^\vee \otimes X^\vee.$$

For bosonic $U(1)$, this is normally a checked boundary and axis-order operation. Other categories may attach phases, twists, pivotal maps, or recoupling transformations.

Relabeling is separate: it changes exact leg compatibility but does not bend, braid, permute, or numerically transform a tensor.

6 Composition and Partial Contraction

Full contraction is morphism composition:

$$A : X \longrightarrow Y, \quad B : Y \longrightarrow Z \quad \Rightarrow \quad B \circ A : X \longrightarrow Z.$$

Ordinary composition checks exact equality of the joined space values, including labels. A named conversion may instead accept structurally compatible spaces and introduce an explicit isomorphism.

Partial contraction is tensor product with identities followed by composition. A checked planner proves this construction and lowers it without materializing identity tensors. Axis positions and explicit contraction pairs determine which occurrences are joined; labels validate the intended pairing but do not address or reorder axes.

Global networks use the same primitives: tensor product, association, recoupling, repartition, symmetric permutation or explicit braid, composition, and categorical trace.

7 BlockTensor Consequences

An illustrative static interface is:

```
template <
    class T,
    class DomainType,
    class CodomainType,
    BlockTensorStorage Storage>
class BlockTensor;

template<class T, CompleteBlockStorage Storage>
using AMatrix = BlockTensor<
    T,
    Domain<BlockSpace, LocalSpace>,
    Codomain<BlockSpace>,
    Storage>;

template<class T, SparseBlockStorage Storage>
using MpoSite = BlockTensor<
    T,
    Domain<LocalSpace, LocalSpace>,
    Codomain<LocalSpace, LocalSpace>,
    Storage>;
```

The static type fixes boundary order, space kinds, and storage requirements. Each tensor value carries the concrete space values for its ordered leg occurrences. Runtime labels distinguish intended bonds, physical legs, impurities, or alternating site types.

A block key remains extensible:

$$\text{BlockKey} = (\text{sector index per boundary occurrence}, \text{CouplingDescriptor}).$$

The coupling descriptor is empty for multiplicity-free $U(1)$, but may identify an invariant channel or fusion basis later.

DenseSpace is load-bearing rather than merely a degenerate case. It permits one block tensor to bundle structurally identical tensors, such as block-Lanczos vectors, a bundled MPS, or tangent vectors. The bundle shares layout metadata and may use coalesced storage.

8 Storage Orthogonality

The same boundary and block keys may use individual host allocations, offsets in coalesced buffers, CUDA-resident blocks, or MPI-distributed blocks. Storage policies choose ownership, allocation, and placement. They preserve:

- all concrete space values and labels;
- domain and codomain order;
- explicit object duality;
- sector indices and coupling descriptors;
- deterministic block keys used by distributed worklists.

Storage movement is not a symmetry transformation. Dense materialization is not an implicit fallback for a symmetry-aware operation.

9 Decompositions Create Bond Values

For

$$A : X \longrightarrow Y,$$

an SVD may use

$$\begin{aligned} V : X \longrightarrow B, \quad S : B \longrightarrow B, \quad U : B \longrightarrow Y, \\ A = U \circ S \circ V. \end{aligned}$$

The decomposition constructs one bond-space value B , chooses its sector structure and label, and places equal copies on the corresponding factor occurrences. Truncation determines B 's final structure before the result is exposed. QR, LQ, polar decomposition, and DMRG bond truncation follow the same value contract.

10 Initial U(1) Implementation Sequence

The first host-only U(1) BlockTensor should:

1. use the concrete implemented space models directly;
2. use the implemented ordered domain and codomain values;
3. add generic `Dual<S>` when its required operations are established;
4. derive block legality from the all-out boundary;
5. keep the block key extensible with an empty coupling descriptor;
6. require exact compatible leg values for ordinary composition;
7. implement the cheap U(1) form of repartition;
8. carry all boundary and space metadata through async worklists, reduced-block kernels, CUDA lowering, and MPI placement.

11 Interface Invariants

1. Concrete types model `Space`; no ownership wrapper is required.
2. Structural space metadata is immutable; the string label is mutable.
3. Exact equality includes the label.
4. `BlockSpace` is canonical; `IrregularSpace` preserves ordered segmentation.
5. `DenseSpace` is symmetry-neutral.
6. Every tensor has an ordered domain and codomain.
7. Object duality is independent of morphism side.
8. Repartition toggles duality and respects order reversal.
9. Labels validate compatibility but never determine axis order.
10. Block selection is an invariant-channel query over the oriented boundary.
11. Global planarity belongs to the network or planner.
12. Storage movement preserves boundary, space, sector, label, and coupling metadata.

11.1 Required tests

The initial tests should include:

- empty symmetric spaces retaining their explicit symmetry;
- canonical unique `BlockSpace` sectors;
- `IrregularSpace` preserving repeated out-of-order blocks and mapping them through explicit regularization;
- `LocalSpace` preserving repeated charges and ordering;
- domain and codomain preserving factor types, values, and order;
- empty domain and codomain values representing the tensor unit;
- exact equality changing when a label changes;
- coordinated bond relabeling updating both endpoints;
- $U(1)$ MPS and MPO block selection from oriented boundaries;
- round-trip repartition and required order reversal;
- decomposition factors receiving equal bond-space values;
- worklists retaining every block key and complete space value.

12 Conclusion

The practical rule is:

Use concrete immutable space structures with mutable semantic labels, orient them through explicit domain/codomain and duality, select legal blocks from that boundary, and leave global planarity to the contraction planner.

For bosonic $U(1)$, these rules compile down to value comparisons and signed charge book-keeping. They are suitable for the first host-only DMRG implementation while preserving the metadata required by CUDA, MPI, non-Abelian fusion, and braiding.